SKJS verifier · research note

Typing Promise Resolution

Native promise contracts and a sound generic Awaited<*T*>, and where TypeScript's own is unsound

Cristiano Calcagno · 8 October 2026

Outline

1. The problem: generic forwarding has no sound type
2. TypeScript's Awaited and why it is unsound
3. Native promise contracts
4. Generic Awaited<*T*>: implied bounds
5. Results: target programs, census, remaining limits

1. Generic forwarding has no sound type

```
async function wrap<T>(x: T): Promise<Awaited<T>> { return x; }
```

The trusted contracts on the base branch:

$$\%\text{Await}\% : \quad \forall T, U \le \text{Prim}.\ (\text{Promise}\langle T\rangle \mid U) \Rightarrow T \mid U$$

$$\%\text{AsyncReturn}\% : \quad \forall T, U \le \text{Prim}.\ (T \mid \text{Promise}\langle U\rangle) \Rightarrow \text{Promise}\langle T \mid U\rangle$$

T may be a promise or an object, and no type says what x becomes once a promise resolves with it. So wrap is refused.

Definitions

Prim: the primitives. Obj: every other value.

Thenable: an object with a callable then.

Resolution: a primitive or non-thenable fulfills the promise; a thenable's then is called.

Promise<*B*>: a native promise whose fulfillment values lie in *B*.

N: values that can never have a callable then.

2. TypeScript's Awaited

```ts
type Awaited<T> =
  T extends null | undefined ? T : // special case
  T extends object & { then(onfulfilled: infer F, ...args: infer _): any } ?
    F extends ((value: infer V, ...args: infer _) => any) ?
      Awaited<V> : // unwrap recursively
    never :
  T; // no declared then
```

- Decided from the *declared* type; distributes over unions; deferred on a type variable.
- Used by Promise.resolve, Promise.all, and to type await x.

Awaited is unsound on object types

Example 1 (object types are open). Accepted by tsc --strict; Node gives "hi".

```
function asRecord(x: { n: number }): { n: number } { return x; }
const t = { n: 1, then(resolve: (v: string) => void) { resolve("hi"); } };
const v = await asRecord(t); // TS: { n: number } Node: "hi"
```

Example 2 (resolution is not idempotent). Accepted by tsc --strict; Node gives 42.

```
const obj = { value: 1 };
const p: Promise<{ value: number }> = Promise.resolve(obj);
Object.assign(obj, { then: (r: (x: number) => void) => r(42) });
const first = await p;     // TS: { value: number } Node: obj
const second = await first; // TS: { value: number } Node: 42
```

Resolution can happen twice

```
const o = { n: 1 };  const p = Promise.resolve(o);  Object.assign(o, { then: (r) => r(42) });
```

Operation on p	Fulfills with
`await p`	the object o
`Promise.resolve(p)`	the object o (it returns p itself)
`Promise.all([p]) element`	the object o
`async () => p`	42
`p.then(), p.then(x => x)`	42
`p.catch(…), p.finally(…)`	42
`new Promise(r => r(p))`	42

Observing a fulfillment value is safe at any payload; resolving it again is safe only if it stays non-thenable.

3. Native promise contracts

Promise<*B*> is a native promise of the realm, with stable standard prototype, constructor, methods and species. A result that is resolved again must stay non-thenable for its whole lifetime:

> *N* = Prim | { readonly then?: Prim } | readonly unknown[]
>
> %Await% : ∀*T*, *U* ≤ *N*. (Promise<*T*> | *U*) ⇒ *T* | *U*
>
> %AsyncReturn% : ∀*T* ≤ *N*, *U* ≤ *N*. (*T* | Promise<*U*>) ⇒ Promise<*T* | *U*>
>
> %PromiseResolve% : ∀*T*, *U* ≤ *N*. (Promise<*T*> | *U*) ⇒ Promise<*T* | *U*>

await admits any promise payload *T*, and a value in *N* at its own type; async return, Promise.resolve of a plain value, catch and finally require *N*.

4. Implied bounds: Awaited<*T*> constrains *T*

```
async function wrap<T>(v: T): Promise<Awaited<T>>
```

> reads as wrap : ∀*T* : Resolvable. (v: *T*) ⇒ Promise<*T*.Result>

- **Implied bound.** Writing Awaited<*T*> in the signature gives *T* the constraint *T* : Resolvable, as if it were written there. Bounds are read off the signature, never off the body.
- ***T* : Resolvable** means *T* has a classification, which fixes two associated types: *T*.Observed, what await yields, and *T*.Result, what a fresh resolution yields, never thenable. Awaited<*T*> is *T*.Result.
- **Not a subtype bound.** It leaves the values of *T* alone; each call supplies the classification of its type argument.
- The other implied bound: a Promise argument of an imported or library signature implies $T \le N$.

Who fixes *T*.Observed and *T*.Result

Rule	type argument *T*	*T*.Observed	*T*.Result
stable	$T \le N$	*T*	*T*
native	Promise<*A*>	*A*	*A*.Result
union	*A* | *B*	*A*.Observed | *B*.Observed	*A*.Result | *B*.Result
narrow	$A \le B$	*B*.Observed	*B*.Result

The producer proposes a classification for each call; Lean checks it and computes *T*.Observed and *T*.Result from it. Inside a body they stay abstract: *T*.Result ≤ *N*, and *T*.Observed resolves to *T*.Result.

Narrow lets *T*.Result be wider than needed: number classifies with result number | string. So *T*.Result is not unique; Awaited corresponds to the least.

Awaited compared with the least *T*.Result

Input type	Awaited	least *T*.Result
primitive, null, undefined	itself	itself (stable)
Promise<number>	number	number (native, stable)
Promise<Promise<number>>	number	number (native twice)
number | Promise<readonly string[]>	number | readonly string[]	number | readonly string[] (union)
{ n: number }	{ n: number }	no classification: refused
PromiseLike<number>	number	no classification: refused
type variable *T*	deferred	*T* : Resolvable, classified at each call

The least *T*.Result agrees with Awaited wherever the result stays non-thenable, and refuses where Awaited is unsound.

5. Target programs

Program	Status
`accept<T>(v: Awaited<T>), a cell of Awaited<T>, wrap<number>`	verifies
`async function wrap<T>(v: T): Promise<Awaited<T>> { return await v; }`	gap: resolution inside a qualified body is refused
`function back<T extends number>(v: Awaited<T>): T`	gap: a result may be wider than T (slide 10)
`wrap<{ n: number }>(v) // open record`	refused
`erase (v: T) => Promise<Awaited<T>> to Promise<unknown>`	refused
`async (v: Promise<{ n: number }>) => v // the 42 case`	refused

All 888 pipeline cases meet their expected outcome; 11 generic Promise cases are recorded gaps.

Census of the SKJS corpus

SKJS corpus, before this work	Units
accepted by SKJS	2674
verified by the verifier	2466
unverified for promise reasons	6

The six promise units	With this work
queryMixedAwaitedLiteralBroadLength: await of an array	verifies (await of a stable value)
testAwaitThenable, testAwaitThenableNested: await of a PromiseLike	unchanged: needs a typed-thenable rule
createCounterService, freshReturnSnapshot: async return of a fresh object	not yet measured
testKeyof: keyof Promise<number>	unchanged: an unrelated restriction

Remaining limits

- **Open:** await and return inside generic bodies, proof search for qualified binders, higher-order uses.
- *T*.**Result is not unique:** narrow admits number with result number | string, so a result bounded by *T* needs a decision, such as taking the least result.
- **Typed thenables:** await of a PromiseLike<*T*> needs a rule for its typed then.
- **Further rules** would cover: returning a fresh object, values a Promise executor resolves, a second reaction on an already-settled promise, thenables that only resolve to themselves, and promises that can never fulfill.
- **Listed gaps:** explicit catch<string>, source Promise copies, Promise<void> reaction receivers, publishing a named generic wrapper.